Problem Set 1 — Report

Summary of “ps1_1.py” (Image processin)

Goal

`ps1_1.py` implements a compact image-processing pipeline to illustrate basic transformations and visual analysis. The pipeline includes resizing, grayscale conversion, Gaussian smoothing, Canny edge detection, binary masking to highlight edges on the color image, and histogram equalization to improve contrast.

Data flow and steps

1. Load the first image found in `input/`.
2. Resize the image to a fixed scale (75% of the original) and save the result.
3. Convert the color image to grayscale and apply Gaussian blur (σ used to determine kernel size). The blurred image is saved.
4. Run Canny edge detection on the grayscale image to produce an edge map. The binary edge image is saved.
5. Create a 3-channel mask from the edge map and blend it with the original color image so edges remain bright while the background is dimmed. Save the masked image.
6. Perform histogram equalization on the grayscale image to boost contrast and save the result.
7. Produce a single combined figure showing the original, intermediate steps, and histograms; save this visualization (the code saves the figure under `output/all_results.png`).

Methods and parameter choices

- **Resizing**: area-preserving interpolation is used for downsampling .
- **Grayscale conversion**: standard color-to-luminance transform.
- **Gaussian blur**: kernel size is computed from σ to ensure an odd kernel and a smooth blur which helps reduce spurious edge detections.
- **Edge detection**: Canny algorithm with two thresholds (low, high). The pair of thresholds controls sensitivity to edges.
- **Masking**: the edge map is converted to a 3-channel mask and blended with the original image so edges remain at full intensity while non-edge pixels are dimmed by a background weight.
- **Histogram** equalization: a global equalization on the grayscale image to redistribute pixel intensity and improve global contrast.

Suggested figures (referenced in Results)

- Grid visualization (`output/all_results.png`): original, resized, grayscale, blurred, edges, masked result, equalized image, original histogram, equalized histogram.
- (output/histograms.png): Close-up comparison: original grayscale histogram vs equalized histogram (highlighting redistribution of intensities).
- Edge mask overlay on color image (cropped sample to show edge fidelity and potential artifacts).

Observations

- Resizing preserved the overall structure and reduced computation for downstream steps.
- Gaussian blur with moderate σ (e.g., σ=2) reduced high-frequency noise while retaining main contours. however, excessive blur removed fine details that could be meaningful edges.
- Canny detection effectively captures strong contours; threshold choices matter — too low produces noisy response, too high misses faint edges. The pipeline used moderately conservative thresholds to produce clear contours for visualization.
- Masking (bright edges, dim background) is visually effective for emphasizing boundaries in cluttered scenes. However, thin edges and small isolated noise may appear disconnected depending on blur and threshold parameters.
- Histogram equalization visibly widened contrast for images with narrow dynamic range; in images with already good contrast, equalization can introduce unnatural quantization or over-amplify noise.

Summary of `p2.py` (Video / live stream processing)

Goal

`p2.py` captures a live video stream (or reads from a provided video file) and produces a side-by-side display of: original frame, masked frame (edges emphasized, background dimmed), and an edge-enhanced frame where edge intensities are blended back into the original, creating an enhanced visualization in real time.

Data flow and steps

1. Open a video source (webcam or `input/1.mp4`).
2. For each frame: convert to grayscale, run Canny edge detection, produce a binary edge mask.
3. Build a masked frame: darken non-edge regions and combine them with full-brightness edge pixels to highlight contours.
4. Edge enhancement: convert the edge mask to a 3-channel image and blend with the original frame to make edges appear stronger.

5. Concatenate the original, masked, and edge-enhanced frames horizontally for display; annotate with FPS and labels.
6. Repeat until the stream ends or the user quits with “q”

Methods and real-time considerations

- The script computes a running FPS estimate and attempts to process frames fast enough to maintain interactive playback.
- To maintain real-time performance, the pipeline uses efficient OpenCV operations (bitwise masking, weighted blending) and avoids costly per-frame allocations where possible.
- Video input fallback: if a webcam is not available, a video file can be used as the input source.

Observations

- Real-time masking and edge boosting provide an immediate visual accentuation of scene structure that can be useful for debugging, artistic effects, or as a preprocessing step for tracking.
- Edge enhancement factor controls how pronounced edges appear; excessive enhancement can make the result look noisy or unnatural.
- On slower hardware, frame-rate drops can lead to reduced interactivity; resolution reduction (downsampling frames) can be used as a practical tradeoff.